

COM2002-1 Programación de Software de Sistemas**Profesor :** Joaquín Soto**Ayudante:** Arnaldo González

Pauta Ayudantía de Repaso

Punteros, strings, memoria dinámica y árboles

Las soluciones de P1 y P2 son una forma de resolver el problema, no la única. Lo importante al corregir es que respeten las restricciones del enunciado (nada de arreglos auxiliares en P1 ni de `string.h` en P2) y que los supuestos queden escritos.

P1. Supuesto: las palabras están separadas por uno o más espacios (' '). Si hay espacios repetidos, al principio o al final, se conservan y simplemente quedan en posición espejada.

La idea es invertir dos veces. Si se invierte la frase completa carácter por carácter, las palabras ya quedan en el orden pedido, pero cada una escrita al revés:

"el proceso padre espera al hijo" → "ojih la arepse erdap osecorp le"

Luego basta recorrer el resultado e invertir cada palabra por separado. Ambos pasos usan la misma función auxiliar, que invierte el tramo entre dos punteros:

```
void invertir_tramo(char *ini, char *fin)
{
    while (ini < fin) {
        char aux = *ini;
        *ini++ = *fin;
        *fin-- = aux;
    }
}

void invertir_palabras(char *frase)
{
    char *fin = frase;
    while (*fin != '\0')
        fin++;
    if (fin == frase)
        return;
    invertir_tramo(frase, fin - 1);

    char *p = frase;
    while (*p != '\0') {
        while (*p == ' ')
            p++;
        char *ini = p;
        while (*p != ' ' && *p != '\0')
            p++;
        invertir_tramo(ini, p - 1);
    }
}
```

Cada carácter se lee y se mueve una cantidad constante de veces (al buscar el final, al invertir la frase y al invertir su palabra), así que el tiempo es $O(n)$, con n el largo de la frase. La memoria adicional es $O(1)$: solo unos punteros y la variable `aux`. Si la palabra termina justo antes de un espacio final, `invertir_tramo` recibe `ini > fin` y no hace nada, por lo que los casos con espacios sobrantes no necesitan tratamiento aparte.

Bonus: rotar k posiciones a la derecha equivale a mover los últimos k caracteres al principio. Se invierte la cadena completa, después los primeros k caracteres y después el resto. Con "sistemas" y $k = 3$:

"sistemas" $\rightarrow$ "sametsis" $\rightarrow$ "masetsis" $\rightarrow$ "massiste"

```
void rotar(char *s, int k)
{
    int n = 0;
    while (s[n] != '\0')
        n++;
    if (n == 0)
        return;
    k = (k % n + n) % n;
    if (k == 0)
        return;
    invertir_tramo(s, s + n - 1);
    invertir_tramo(s, s + k - 1);
    invertir_tramo(s + k, s + n - 1);
}
```

Rotar n posiciones deja la cadena igual, por eso se trabaja con k módulo n . La suma de n antes del segundo módulo hace que un k negativo se interprete como rotación a la izquierda (en C, $-5 \% 8$ da -5). Son tres pasadas lineales, así que el tiempo es $O(n)$ y no se usa memoria extra.

- P2.** a) Primero se calculan los dos largos con una función propia, porque no se puede usar `strlen`. Después se corren hacia la derecha, en `largo(src)` posiciones, todos los caracteres de `dest` desde `pos` hasta el `'\0'` incluido, y por último se copia `src` en el hueco que queda.

```
size_t largo(const char *s)
{
    const char *p = s;
    while (*p != '\0')
        p++;
    return p - s;
}
```

```

char *strins(char *dest, const char *src, size_t pos)
{
    size_t ld = largo(dest);
    size_t ls = largo(src);

    if (pos > ld)
        pos = ld;
    if (ls == 0)
        return dest;

    for (size_t i = ld + ls; i >= pos + ls; i--)
        dest[i] = dest[i - ls];
    for (size_t i = 0; i < ls; i++)
        dest[pos + i] = src[i];
    return dest;
}

```

El primer ciclo parte en `ld + ls`, que es donde termina el nuevo `'\0'`. Dejar `pos = ld` cuando `pos` es mayor cumple lo que pide el manual: `src` se agrega al final. El `return` cuando `src` es vacío no es solo un atajo: con `ls == 0` y `pos == 0` la condición sería `i >= 0`, que con `size_t` siempre es verdadera, y el ciclo nunca terminaría. Es un error típico al recorrer hacia atrás con índices sin signo.

b) La versión del compañero sería esta:

```

for (size_t i = pos; i <= ld; i++)
    dest[i + ls] = dest[i];

```

Con el ejemplo del enunciado funciona. Hay que mover "pointer" y su `'\0'` (8 caracteres, posiciones 8 a 15) hacia las posiciones 16 a 23. Como el desplazamiento también es 8, ninguna escritura cae sobre un carácter que falte por leer.

El problema aparece cuando el tramo que hay que mover es más largo que `src`. Por ejemplo, con `dest = "abcdef"`, `src = "XY"` y `pos = 1`, el desplazamiento es 2 y el ciclo va escribiendo sobre posiciones que todavía no ha leído:

	0	1	2	3	4	5	6	7	8
inicio	a	b	c	d	e	f	\0		
i = 1	a	b	c	b	e	f	\0		
i = 2	a	b	c	b	c	f	\0		
i = 3	a	b	c	b	c	b	\0		
i = 4	a	b	c	b	c	b	c		
i = 5	a	b	c	b	c	b	c	b	
i = 6	a	b	c	b	c	b	c	b	c

En `i = 3` se lee la `b` que se acababa de escribir en vez de la `d` original, y desde ahí el patrón `bc` se repite. Además, el `'\0'` se sobrescribe en `i = 4` antes de haberlo copiado, así que la cadena resultante no queda terminada. Después de copiar `src` se obtiene `"aXYbcbcbc"` si el resto del arreglo tenía ceros, o basura a continuación si no. Recorriendo de derecha a izquierda, cada posición se lee antes de ser pisada, que es la misma razón por la que `memmove` elige la dirección de copia según cómo se solapan los bloques.

Bonus: hay que suponer que `dest` fue pedido con `malloc`, `calloc` o `realloc`. Si fuera un arreglo local o un literal, `realloc` no se puede usar sobre él.

```
char *strins2(char *dest, const char *src, size_t pos)
{
    char *nuevo = realloc(dest, largo(dest) + largo(src) + 1);
    if (nuevo == NULL)
        return NULL;
    return strins(nuevo, src, pos);
}
```

Si `realloc` logra agrandar el bloque en el mismo lugar, retorna el mismo puntero. Si no, copia el contenido a otro bloque, libera el antiguo y retorna la nueva dirección, y por eso quien llama tiene que quedarse con el valor retornado. Si falla, retorna `NULL` sin tocar el bloque original, y conviene conservarlo para poder liberarlo:

```
char *msg = malloc(16);
strcpy(msg, "free(): pointer");

char *aux = strins2(msg, "invalid ", 8);
if (aux == NULL) {
    free(msg);
    return 1;
}
msg = aux;
printf("%s\n", msg);
free(msg);
```

Escribir directamente `msg = strins2(msg, ...)` funciona mientras no haya error, pero si `realloc` falla se pierde la única referencia al bloque original y queda una fuga de memoria.

- P3.** a) `sizeof(a)` vale 7, porque `a` es un arreglo con las 6 letras más el `'\0'`. `sizeof(p)` vale 8, porque `p` es un puntero y `sizeof` mide el puntero, no lo apuntado. `strlen` da 6 en los dos casos, porque solo cuenta hasta el `'\0'`.
- `a[0] = 'K'` es válida: `a` es una copia del literal guardada en la pila de la función. `p[0] = 'K'` compila, pero intenta modificar el literal, lo que es comportamiento indefinido. En la práctica los literales quedan en una zona de solo lectura y el programa termina con *segmentation fault*. Lo correcto es declarar `const char *p`, y así el compilador rechaza la asignación.
- b) Imprime 8. Un parámetro declarado como `char s[]` es exactamente un `char *s`: al pasar un arreglo a una función se pasa la dirección de su primer elemento y el tamaño se pierde. gcc incluso lo advierte con `-Wsizeof-array-argument`. Para `strins()` esto significa que no hay forma de revisar desde dentro si `dest` tiene espacio suficiente, y por eso el manual deja esa responsabilidad a quien llama.
- `const char *src` es un puntero a caracteres constantes: `*src = 'x'` no compila, pero `src++` sí es válido, porque el puntero en sí puede cambiar (y es cómodo para recorrer). Con `char *const src` es al revés: los caracteres se pueden modificar, pero el punte-

ro no se puede reasignar ni incrementar. El `const` del encabezado sirve también como documentación: le promete a quien llama que `src` no se va a modificar.

- c) La función es $O(n^2)$. `strlen` recorre la cadena completa, $O(n)$, y la condición del `for` se evalúa en cada una de las $n + 1$ vueltas. Basta con calcular el largo una vez antes del ciclo o, más simple, cambiar la condición a `s[i] != '\0'`, con lo que queda $O(n)$.

Con `strcat` pasa lo mismo de forma menos visible: cada llamada recorre `buf` desde el inicio para encontrar su `'\0'`. Si las k palabras miden m en promedio, la llamada i recorre unos $i \cdot m$ caracteres ya escritos, y el total es del orden de $m k^2/2$. Se evita recordando dónde termina lo ya escrito:

```
char *fin = buf;
for (int i = 0; i < k; i++) {
    const char *p = palabras[i];
    while (*p != '\0')
        *fin++ = *p++;
}
*fin = '\0';
```

- d) Si `izq` fuera de tipo `struct nodo`, cada nodo contendría otro nodo completo, que a su vez contendría otro, y la estructura tendría tamaño infinito. Además, en ese punto el tipo todavía está incompleto y gcc lo rechaza (*field has incomplete type*). Un puntero, en cambio, siempre mide 8 bytes, sin importar a qué apunte, y permite representar la ausencia de hijo con `NULL`.

`sizeof(struct nodo)` vale 24 y no 20 por el alineamiento: un puntero de 8 bytes debe quedar en una dirección múltiplo de 8, así que el compilador agrega 4 bytes de relleno (*padding*) después de `valor`. El tamaño total también queda múltiplo de 8, para que un arreglo de nodos mantenga alineados todos sus elementos.

`r->izq` y `(*r).izq` son lo mismo: la flecha es una abreviación de desreferenciar y luego acceder al campo. `*r.izq` se lee como `*(r.izq)`, porque el punto tiene mayor precedencia que el asterisco, y no compila, ya que `r` es un puntero y no una estructura.

- e) I. Falta el espacio del `'\0'`. `strcpy` también copia el terminador, y ese último byte cae fuera del bloque (AddressSanitizer lo reporta como *heap-buffer-overflow*). Además conviene revisar el retorno de `malloc`:

```
char *d = malloc(strlen(s) + 1);
if (d == NULL)
    return NULL;
strcpy(d, s);
```

II. Se ignora el valor retornado por `realloc`. Si el bloque se mueve, `s` queda apuntando a memoria ya liberada y el `strcat` escribe sobre ella (*heap-use-after-free*). Si `realloc` falla, retorna `NULL`. Por eso se guarda el resultado en un auxiliar:

```

char *aux = realloc(s, 11);
if (aux == NULL) {
    free(s);
    return 1;
}
s = aux;
strcat(s, " mundo");

```

III. `n` es una variable local: vive en el marco de la función y deja de existir al retornar, así que el puntero devuelto queda colgando. gcc lo advierte con `-Wreturn-local-addr`. El nodo tiene que vivir en el heap:

```

struct nodo *nuevo_nodo(int v)
{
    struct nodo *n = malloc(sizeof(struct nodo));
    if (n == NULL)
        return NULL;
    n->valor = v;
    n->izq = n->der = NULL;
    return n;
}

```

f) `misterio` cuenta las hojas del árbol, es decir, los nodos sin hijos. Un árbol vacío tiene 0 hojas y un nodo sin hijos es una hoja. En cualquier otro caso, las hojas del árbol son las del subárbol izquierdo más las del derecho.

Para liberar hay que recorrer en postorden: primero los dos subárboles y al final el nodo. Si se hiciera `free(r)` antes, ya no se podría leer `r->izq` ni `r->der`, porque acceder a un bloque liberado es comportamiento indefinido.

```

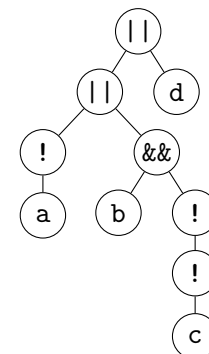
void liberar(struct nodo *r)
{
    if (r == NULL)
        return;
    liberar(r->izq);
    liberar(r->der);
    free(r);
}

```

g) I. Como `||` tiene la menor precedencia y asocia a la izquierda, la expresión se agrupa como

`((!a) || (b && !(c))) || d`

La raíz es el segundo `||`, y los nodos con un solo hijo son los tres `!`: el que niega a `a` y los dos que están sobre `c`.



- II. El postorden es $a \ ! \ b \ c \ ! \ ! \ \&\& \ || \ d \ ||$. Es el orden adecuado para evaluar porque cada operador aparece después de sus operandos, así que al llegar a un nodo ya se conocen los valores de sus hijos. Es la notación polaca inversa: se puede evaluar de izquierda a derecha con una pila, apilando variables y aplicando cada operador a los valores del tope.
- III. Regla: la raíz es el operador de menor precedencia que aparece en la expresión. Si aparece varias veces, se elige el que está más a la derecha, porque $\&\&$ y $||$ asocian a la izquierda y ese operador es el último que se evalúa. Si no hay operadores binarios y la expresión empieza con $!$, la raíz es ese $!$ y su único hijo es el resto de la expresión. Algoritmo: si la expresión es una sola variable, se retorna una hoja. Si no, se busca la raíz con la regla anterior, primero entre los $||$ y luego entre los $\&\&$. Se crea un nodo con ese operador, y sus hijos se obtienen aplicando recursivamente el mismo procedimiento al texto de la izquierda y al de la derecha. En el caso de $!$ se aplica a lo que viene después.
- IV. Con paréntesis, la regla solo considera operadores que no estén dentro de ningún paréntesis. Para eso se lleva un contador de profundidad que sube con $($ y baja con $)$, y se buscan operadores solo donde el contador es 0. Se agrega un caso: si no hay operadores a profundidad 0 y la expresión empieza con $($, entonces está completa entre paréntesis, que se quitan antes de seguir.

En $!(a \ || \ b) \ \&\& \ c$ el único operador a profundidad 0 es $\&\&$, que queda de raíz. A la izquierda queda $!(a \ || \ b)$: ahí no hay binarios a profundidad 0 y empieza con $!$, así que su raíz es ese $!$, y su único hijo es $(a \ || \ b)$, que se trata quitando los paréntesis. El postorden es $a \ b \ || \ ! \ c \ \&\&$.

